

SmartExpense: Event-Aware Expense Tracker with AI-Powered Spend Prediction

Project Proposal — UCS503P

Submitted to: Mr. Asif

Thapar Institute of Engineering and Technology

Navya Arora (1024030438), Divanshi Garg (1024030441)

August 2026

1. Introduction and Problem Statement

Personal finance tools are mostly passive, i.e. they record past expenses and show historical charts but provide little guidance for future spending. Predictable events such as birthdays, anniversaries, and festivals can still cause unexpected financial pressure because existing tools rarely connect upcoming events with spending patterns.

The main gaps are:

- No event-aware forecasting of future expenses.
- Fragmented planning, with budgets, notes and reminders handled in different apps.
- Manual expense entry, which can be time-consuming and inconvenient.

SmartExpense addresses these gaps through an integrated platform that combines expense tracking, event-aware spending prediction, reminders, and a Notes-style planning workspace. It aims to transform expense tracking from a passive record-keeping tool into a proactive financial planning system.

2. Objectives

Primary Goal: To design, build, and deploy a full-stack web application that predicts future-month personal expenditure per category, adjusted for user-registered life events, and notifies users proactively before those events occur.

- To implement a secure MERN-stack expense tracking system with JWT-based authentication, transaction CRUD, and keyword-based categorization supporting both manual entry and bulk CSV upload.
- To build a Python/FastAPI microservice that fits a Prophet time-series model on per-user transaction history, with registered events as binary regressors, and exposes a /predict-spend endpoint called monthly by a scheduled job.
- To implement an event registry module that stores recurring and one-time life events, links them to spending categories, and triggers proactive email reminders (Nodemailer) within 7 days of each event.
- To deliver an integrated Notes and Saved Items module where users can plan for events.
- To deploy all three services (React frontend, Express backend, FastAPI ML service) to production on free-tier cloud infrastructure (Vercel, Render, MongoDB Atlas) with a live accessible URL.

3. Methodology

System architecture is defined upfront, features are built in three sequential iterations with deployment after each, and the ML layer is validated against measurable accuracy targets before being integrated into the product.

3.1 System Architecture and Tech Stack

The application is a three-service architecture:

Layer	Technology	Role
Frontend	React (Vite) + Tailwind CSS + Recharts	UI, routing, data visualization
Backend API	Node.js + Express + JWT + bcrypt + node-cron	Auth, CRUD, scheduling, orchestration
ML Service	Python + FastAPI + Prophet + scikit-learn	Spend forecasting, anomaly detection
Database	MongoDB Atlas + Mongoose	All persistent storage

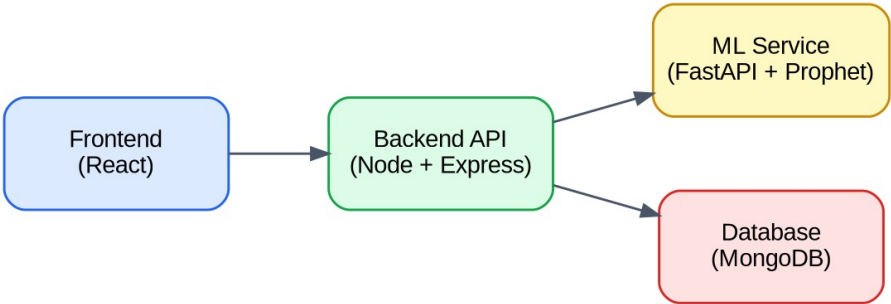


Figure 1: High-level system architecture.

3.2 Module Development

Auth Module: bcrypt password hashing, JWT-based stateless auth, protected routes on all data endpoints.

Transactions Module: Full CRUD for transactions with amount, date, merchant, and category fields. CSV bulk upload (Multer) parses bank statement exports; a keyword-matching function (e.g., “swiggy/zomato” : Food, “uber/ola” : Transport) assigns categories automatically, covering 80% of transactions without manual tagging.

Events Module: Users register recurring or one-time events (name, date, recurrence, linked category, optional expected spend). MongoDB stores event history including actual post-event spend for model learning.

AI/Prediction Service (FastAPI): On a monthly cron trigger, Express calls POST /retrain-and-predict with the user’s transaction history. FastAPI aggregates transactions to monthly category totals, adds binary event regressors for the upcoming month, and fits a Prophet model. The resulting spend prediction per category is stored in a Predictions collection in MongoDB. A z-score threshold on individual transactions flags anomalies for display in the dashboard.

Notes and Saved Items Module: Freeform notes (text, checkboxes, pinning) linked to events. Saved item costs are summed and passed as an additional input to the prediction service (if users click on the option of adding to the spendings).

3.3 Data Collection and Model Training

Training data is sourced from the user’s own transaction history, entered via manual logging or CSV upload from their bank or UPI app statement. For the initial development and pilot phase, the developer’s own anonymized 3-month bank statement will seed the system. A cold-start fallback (category averages from the first 1-2 months) handles new users before sufficient history exists to fit a reliable model. Monthly cron-triggered per-user retraining ensures the model improves as transaction history grows.

3.4 Testing and Validation

Each module will be validated with Postman API tests before integration. End-to-end testing will cover the critical path shown below: CSV upload, categorization, event registration, prediction trigger, reminder email delivery.

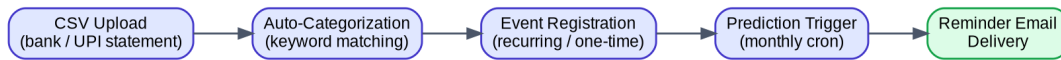


Figure 2: End-to-end process flow, from CSV upload to reminder delivery.

The primary ML accuracy metric is Mean Absolute Error (MAE) between predicted and actual monthly category spend, measured at month-end from database records. Target: median MAE ≤ 500 per category after 3 months of data.

3.5 Risks and Mitigations

- Cold-start (no history for new users): category-average fallback, communicated clearly in the UI; CSV import at onboarding accelerates history collection.
- Manual entry abandonment: weekly “you haven’t logged in X days” nudge via the existing reminder infrastructure; CSV upload as a frictionless alternative.
- Render free-tier concurrent retraining load: stagger monthly cron jobs across users rather than triggering all at midnight on the 1st.

4. Timeline

Week	Phase	Tasks	Deliverable
3-4	Setup & Planning	Finalize requirements, set up GitHub repository, create MERN project structure, connect MongoDB, and design database schema	Working project setup and database connection
5-6	Expense Module	Implement expense CRUD, categories, transaction forms, and basic dashboard/charts	Functional expense tracking module
7-8	Events & Reminders	Implement event CRUD, recurring events, and scheduled reminders using node-cron	Functional event and reminder system
9-10	AI/ML Module	Set up Python/FastAPI service, prepare transaction data, develop initial forecasting model, and integrate it with the backend	Initial spending prediction integrated with dashboard
11	Integration & Testing	Integrate all modules, test APIs, fix bugs, improve UI, and evaluate prediction results	Integrated and tested application
12	Finalization & Deployment	Final testing, documentation, deployment, and presentation preparation	Final working application and project documentation

5. Resources and Budget

Human Resources: Group of two project.

- ML prediction of future spendings and ideation handled by Navya Arora.
- MERN stack application and ideation is handled by Divanshi Garg.

Software and Tools: All tools are free and open-source — React, Node.js, Express, Python, FastAPI, Prophet, scikit-learn, MongoDB Atlas (free tier), Vercel (free tier), Render (free tier), Postman (free), GitHub.

External APIs: Gmail SMTP via Nodemailer for reminder emails (free).

Budget: No external funding required. All hosting, database, and API services operate within free tiers appropriate for a portfolio-scale educational deployment. No hardware beyond a personal laptop is needed.

6. Summary

The Gap: Existing expense trackers are passive; they record spending but do not connect it to a user's calendar of predictable life events, leaving users financially unprepared for costs they could have anticipated.

The Solution: SmartExpense is a three-service MERN + Python web application that forecasts next-month spending per category using predictive time-series models with event-based regressors, sends proactive reminders ahead of registered events, and provides an integrated notes workspace for event planning.

The Impact: Users shift from reactive expense logging to proactive financial planning — knowing before a birthday month arrives what to budget, where to shop, and how much they typically spend.